

EventMesh — After Step 5: Short Study Guide

Step 6 onward | Hinglish | WHAT → WHY → HOW → FLOW → COMMAND → VERIFY

Goal: Step 5 ke baad jo build hoga uska mental model samajhna. Commands ratne ke bajay connection aur reason samjho.

1. Current Architecture

Current: Express API → Prisma → PostgreSQL. PostgreSQL Docker container mein hai. Later: API → Redis/BullMQ → Worker → Consumer Webhook.

WHY: PostgreSQL durable business data rakhta hai; Prisma TypeScript aur DB ke beech data-access layer hai; Docker reproducible local infrastructure deta hai; Redis/BullMQ slow/retryable work ko asynchronous banayega.

2. Docker PostgreSQL — WHAT / WHY / HOW

WHAT: PostgreSQL 17 container ke andar run ho raha hai. **WHY:** local environment isolated/reproducible rahe. **HOW:** Compose image, credentials, database, port aur volume define karta hai.

```
postgres:17
user = eventmesh
password = eventmesh_dev
database = eventmesh
5432:5432
postgres_data:/var/lib/postgresql/data
```

5432:5432: HOST port 5432 → CONTAINER port 5432. Isliye localhost:5432 se Docker PostgreSQL tak pahunchte hain.

Volume: database files container lifecycle se alag persist hoti hain. Volume delete karna destructive ho sakta hai.

3. Docker Commands — What They Do

```
docker compose up -d postgres
# PostgreSQL service start

docker ps
# running containers

docker logs eventmesh-postgres
# container logs

docker exec eventmesh-postgres pg_isready -U eventmesh -d eventmesh
# PostgreSQL ready hai?

docker exec -it eventmesh-postgres psql -U eventmesh -d eventmesh
# DB shell ke andar jao
```

Flow: Compose → container → PostgreSQL process → port 5432 → host application.

4. Real Debugging Lesson — Port Conflict

Windows par native PostgreSQL 18 bhi port 5432 use kar raha tha. Docker PostgreSQL bhi 5432 publish kar raha tha. Host connection wrong PostgreSQL instance tak ja raha tha.

```
Windows PostgreSQL 18
↓
port 5432 ← conflict

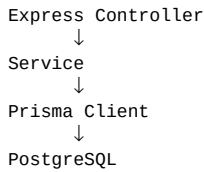
Docker PostgreSQL 17
↓
port 5432
```

Useful checks: `Get-NetTCPConnection -LocalPort 5432 -State Listen` → kaun port use kar raha hai; `Get-Process` → process; `Get-Service` → Windows service. Conflicting dev service ko appropriate hone par stop kiya: `Stop-Service postgresql-x64-18`.

Debug principle: network → port owner → authentication → database state → ORM configuration, ek-ek layer isolate karo.

5. Prisma — WHAT / WHY / FLOW

WHAT: Prisma TypeScript application ka ORM/data-access layer hai. **WHY:** type-safe DB access, schema management aur migrations structured hote hain.



Core concepts: schema = desired database structure; migration = schema change ko database mein apply karna; Prisma Client = application se database query karna.

6. Prisma 7 Config + DATABASE_URL

Prisma 7 setup mein `prisma7.config.ts` schema, migration path aur database URL configure karta hai. `dotenv/config.env` load karta hai.

```

import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: { path: "prisma/migrations" },
  datasource: { url: process.env["DATABASE_URL"] },
});
  
```

Connection flow: `.env` → **DATABASE_URL** → `prisma7.config.ts` → **Prisma** → **PostgreSQL**.

`postgresql://eventmesh:eventmesh_dev@localhost:5432/eventmesh` = protocol / username / password / host / port / database.

7. Prisma Commands

```

npx prisma validate
# schema/config valid?

npx prisma db pull
# EXISTING database → Prisma schema

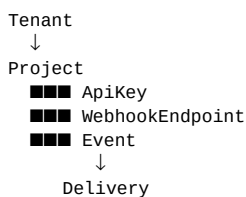
npx prisma migrate dev --name init
# Prisma schema → database migration

npx prisma generate
# Prisma Client generate/update
  
```

Critical: db pull tables create nahi karta. EventMesh mein models define karke `migrate dev` use karenge.

Current P4001: Prisma database tak successfully connect hua, but database mein zero tables the. Iska matlab authentication/connection issue nahi tha.

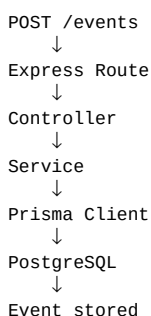
8. Database Design — Next



Tenant: customer/account. **Project:** tenant ka application/environment. **ApiKey:** publisher authentication. **WebhookEndpoint:** consumer URL. **Event:** published event. **Delivery:** one event → one endpoint delivery state.

Important: one Event can create many Deliveries. Example: one `payment.success` + 3 endpoints = 3 independent delivery records.

9. Complete Request/Data Flow



Later:
Event → delivery jobs
→ Redis/BullMQ
→ Worker
→ HTTP webhook
→ Consumer

WHY asynchronous? Consumer slow/down ho sakta hai. API ko webhook response ka wait nahi karna chahiye. Queue + worker retries, delays aur failure handling provide karenge.

10. Exact Learning Order From Here

1) Entity relationships → 2) schema.prisma → 3) validate → 4) migration → 5) inspect tables → 6) Prisma Client → 7) service layer → 8) event API → 9) Redis → 10) BullMQ producer → 11) worker → 12) webhook delivery → 13) retry/DLQ.

11. Interview Mental Model

HTTP receives request → middleware handles cross-cutting concerns → controller handles HTTP boundary → service owns application operation → Prisma accesses PostgreSQL → PostgreSQL stores durable state → Redis/BullMQ handles asynchronous jobs → worker performs slow/retryable delivery.

Golden principle: synchronous API path fast/predictable rakho; slow or retryable work asynchronous workers ko do.

12. Before Continuing, You Should Be Able To Explain

■ 5432:5432 kya hai? ■ Docker PostgreSQL kyu? ■ Native PostgreSQL 18 conflict kyu hua? ■ DATABASE_URL ka har part kya hai? ■ Prisma ka role kya hai? ■ db pull vs migrate dev? ■ P4001 ka meaning? ■ Event vs Delivery? ■ API → DB → Queue → Worker → Webhook flow?